

Appunti di Basi di dati 1

Emanuele Romano

Indice

1	Progettazione Concettuale	1
1.1	Progettazione Concettuale con Class Diagrams	1
1.1.1	Ruoli nelle Associazioni	2
1.1.2	Navigabilità e Direzionalità	2
1.1.3	Tipi Enumerativi (Enumerazioni)	3
1.1.4	Classi di Associazione	3
1.1.5	Grado di un'Associazione e Relazioni Ternarie	5
1.1.6	Aggregazione e Composizione	7
1.1.7	Generalizzazioni/Specializzazioni	8
1.2	La Documentazione di Supporto: I Dizionari dei Dati	9
2	Modello Relazionale e Progettazione Logica	11
2.1	Introduzione	11
2.2	Concetti Fondamentali	11
2.3	Le Chiavi	13
2.4	Traduzione delle Associazioni	14
2.4.1	Associazioni 1 a 1	14
A	Bozze	16
A.1	Aspetti implementativi dell'ereditarietà	16
A.2	lezione 7	17

Progettazione Concettuale

1.1 Progettazione Concettuale con Class Diagrams

La progettazione concettuale è la fase in cui si modella il *cosa* deve essere rappresentato nella base di dati, ignorando i dettagli implementativi del DBMS. Sebbene la letteratura classica utilizzi il modello Entity-Relationship (ER), l'approccio moderno predilige l'uso dei **Class Diagrams UML**, di cui di seguito richiameremo i concetti fondamentali. Si assumerà che il lettore abbia già familiarità con i diagrammi UML in un contesto orientato agli oggetti, dunque ne faremo dei richiami piuttosto sommari e ci concentreremo sulle specificità della modellazione per basi di dati.

Il punto di partenza è un testo in linguaggio naturale (specifiche). La costruzione del diagramma segue una strategia di mapping semantico:

- **Sostantivi → Classi (Entità):** Rappresentano oggetti con esistenza autonoma (es. *Studiante, Corso*).
- **Verbi → Associazioni (Relazioni):** Descrivono i legami logici tra le classi (es. *Iscrive, Sostiene*).
- **Aggettivi o Specifiche → Attributi:** Proprietà atomiche che descrivono una classe (es. *Nome, Matricola*).

Gli elementi costitutivi di un Class Diagram sono:

Classe: Rappresentata da un rettangolo. Nella progettazione concettuale ci si concentra su *Nome* e *Attributi*. Le *Operazioni* (metodi) vengono solitamente omesse.

Associazione: Una linea che connette due classi. Indica che esiste una correlazione logica tra le istanze delle classi coinvolte.

Cardinalità: Indicano il numero minimo e massimo di istanze di una classe che possono essere legate a una singola istanza della classe opposta.

Tabella 1.1: Sintassi delle Cardinalità UML

Sintassi	Significato
1	Esattamente uno (Obbligatorio)
0..1	Zero o uno
* o 0..*	Zero o molti
1..*	Uno o molti (Almeno uno)

Un attributo si dice **totale** se ha cardinalità minima strettamente maggiore di zero, **parziale** altrimenti: quindi il primo deve sempre avere un valore, mentre per uno parziale il valore è opzionale.

Un attributo si dice **singolo** se ha cardinalità massima uguale a uno, **multiplo** altrimenti.

Un attributo è **atomico** se il database non ne percepisce la struttura interna. Gli attributi atomici possono essere:

- **Primitivi:** Rappresentano valori elementari non scomponibili.

- *Numerici*: `int`, `float`, `decimal`.
- *Alfanumerici*: `string`, `char`.
- *Logici*: `boolean`.
- **Predefiniti Strutturati**: Tipi di dato complessi forniti dal sistema e trattati come atomici dal DBMS.
 - *Temporal*: `date`, `time`, `timestamp`.
 - *Oggetti Binari*: `blob` (immagini, file), `clob` (testi estesi).

Un errore comune è modellare come attributo ciò che dovrebbe essere una classe. La regola empirica è:

1. Se l'informazione è un valore semplice (es. *Età*), è un **Attributo**.
2. Se l'informazione ha proprietà proprie o deve essere legata ad altre entità (es. *Indirizzo* inteso come via, civico e CAP), deve essere una **Classe** autonoma.



L'insieme degli attributi deve permettere di identificare univocamente ogni istanza della classe.

Osservazione 1.1. Sebbene in Java usi `-` (`private`) e `+` (`public`), nei diagrammi per database la visibilità è meno rilevante, ma mantenere il `-` per gli attributi aiuta a ricordare l'incapsulamento dei dati.

1.1.1 Ruoli nelle Associazioni

Un **ruolo** identifica la funzione specifica che un'istanza di una classe svolge all'interno di un'associazione. Viene indicato alle estremità della linea di associazione.

- **Utilità**: I ruoli sono essenziali per chiarire il significato del legame, specialmente quando l'associazione è *riflessiva* (connette una classe a sé stessa) o quando esistono più associazioni tra le stesse classi.
- **Naming Convention**: Solitamente si utilizza un sostantivo che descrive la responsabilità (es. *responsabile*, *supervisore*, *partecipante*).
- **Corrispondenza OOP**: Nella programmazione ad oggetti, il nome del ruolo coincide spesso con il nome dell'attributo (o del campo) che contiene il riferimento all'oggetto correlato.

Esempio 1.1 (Associazione Riflessiva o Ricorsiva). Si consideri la classe **Persona** e l'associazione *Genitorialità*. Senza ruoli, il legame sarebbe ambiguo. Assegnando i ruoli **genitore** e **figlio** alle due estremità, la semantica del modello diventa univoca.

Osservazione 1.2. Spesso si tende a omettere il nome dell'associazione (il verbo al centro della linea) se i ruoli sono già sufficientemente esplicativi. In molti schemi professionali vengono mostrati solo i ruoli, perché sono più vicini alla struttura logica finale del database.

1.1.2 Navigabilità e Direzionalità

Una differenza cruciale tra la modellazione OOP e quella per basi di dati riguarda la **navigabilità** delle associazioni.

- **In Java (OOP)**: Le associazioni sono spesso *unidirezionali* per favorire il disaccoppiamento tra le classi. La direzione indica quale oggetto mantiene il riferimento (puntatore) verso l'altro.

- **Nelle Basi di Dati:** Le associazioni sono intrinsecamente *bidirezionali*. Una relazione tra due entità rappresenta un fatto logico che può essere interrogato partendo da entrambi i lati (es. tramite operatori di *Join* in SQL).

Dunque nel class diagram concettuale per database, si evitano solitamente le frecce di navigabilità. Una linea semplice indica che l'informazione è disponibile per l'interrogazione in modo simmetrico. La "direzionalità" fisica verrà stabilita solo nella fase di progettazione logica attraverso il posizionamento delle chiavi esterne.

1.1.3 Tipi Enumerativi (Enumerazioni)

Quando il dominio di un attributo è costituito da un insieme di valori costante, discreto e predefinito, si utilizza il costrutto della **Enumerazione**. Si tratta di un tipo di dato speciale che permette a un attributo di assumere solo uno dei valori elencati in una lista chiusa.

In UML, le enumerazioni sono rappresentate come classi con lo stereotipo «enumeration». Non hanno attributi propri, ma contengono un elenco di valori letterali che rappresentano le possibili istanze del tipo enumerativo.



A differenza dei diagrammi in OOP, le enumerazioni **non sono classi**, ma semplicemente un tipo di dato speciale; **non hanno associazioni con le classi**.

1.1.4 Classi di Associazione

Nella modellazione concettuale, capita spesso che un'associazione tra due classi possieda delle proprietà proprie. Se un attributo non descrive intrinsecamente né la classe A né la classe B, ma esiste solo in funzione della loro relazione, ci troviamo di fronte alla necessità di una **Classe di Associazione**.

Si consideri, ad esempio, il caso classico di un sistema universitario: uno **Studente** sostiene un **Corso**. Vogliamo memorizzare il *voto* conseguito.

- Il *voto* non è un attributo dello **Studente** (egli ha voti diversi per corsi diversi).
- Il *voto* non è un attributo del **Corso** (il corso ha voti diversi per studenti diversi).

In un diagramma standard, saremmo costretti a omettere il voto o a inserirlo forzatamente in una delle due classi, creando un errore concettuale e ridondanza.

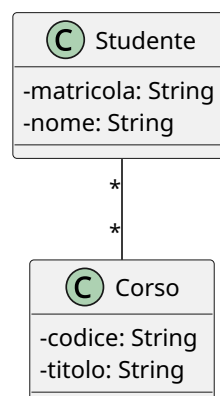


Figura 1.1: Associazione Studente-Corso.

L'esplicitazione è spesso necessaria perché molti strumenti di modellazione o linguaggi di programmazione non supportano nativamente il costrutto della classe di associazione. In questo caso, la classe intermedia (es. **Esame**) funge da "ponte" tra le due entità principali.

💡 Tip

La classe di associazione è significativa solo su associazioni *multi-a-multi*, ma occasionalmente si usa anche per altri tipi di associazioni.

1.1.5 Grado di un'Associazione e Relazioni Ternarie

Il **grado** di un'associazione è definito come il numero di classi coinvolte nel legame logico.

- **Associazioni Binarie (Grado 2):** Collegano due classi. Costituiscono la quasi totalità delle relazioni in un database.
- **Associazioni N-arie (Grado $N \geq 3$):** Collegano tre o più classi contemporaneamente. Il caso più comune è l'associazione **ternaria**.

Un'associazione ternaria si rende necessaria quando il legame logico esiste solo se si considerano tre entità simultaneamente; la scomposizione in relazioni binarie separate causerebbe una perdita irreparabile di informazione semantica. In UML si rappresenta graficamente interponendo un **rombo** tra le linee che collegano le tre classi.

Si consideri, ad esempio, uno scenario di logistica industriale in cui sono coinvolte tre classi: **Fornitore**, **Prodotto** e **Progetto** (cantiere di destinazione). L'atto del rifornimento è un evento atomico: il *Fornitore A* fornisce il *Prodotto X* specificamente per il *Progetto Y*. Se provassimo a mappare questo dominio con tre relazioni binarie (*Fornitore-Prodotto*, *Prodotto-Progetto*, *Fornitore-Progetto*), sapremmo quali prodotti vende un fornitore e quali prodotti usa un progetto, ma non sapremmo mai **quale specifico fornitore ha portato quel determinato prodotto a quel singolo progetto**.

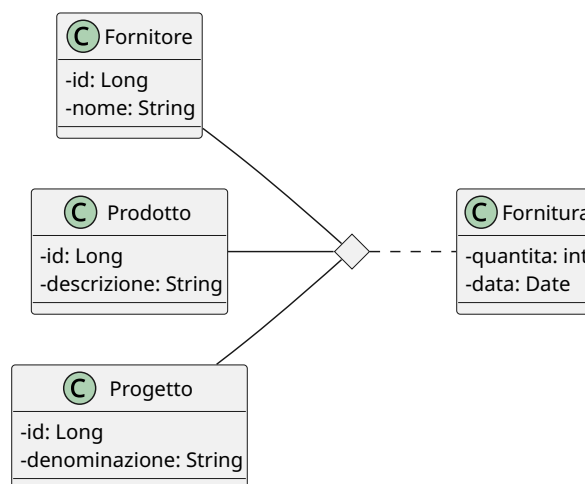


Figura 1.4: Esempio associazione ternaria con classe di associazione.

💡 Interpretazione di una associazione n-aria

Per comprendere a fondo l'associazione ternaria, si può pensare che una delle tre classi non si colleghi singolarmente alle altre due, ma alla loro **coppia**. Nel

🔗 nostro esempio, il **Progetto** non è legato separatamente a un **Prodotto** e a un **Fornitore**, ma è associato stabilmente alla coppia (*Prodotto*, *Fornitore*), intesa come specifica combinazione. Analogamente per le altre due classi.

Poiché i DBMS relazionali e i framework ORM (come JPA/Hibernate) non supportano le relazioni native a tre vie, si deve **esplicitare** (o reificare) l'associazione prima dell'implementazione.

La reificazione trasforma il rombo dell'associazione in una classe vera e propria, che chiameremo **AssegnazioneFornitura**. L'associazione ternaria originale viene così frantumata in tre associazioni binarie distinte di tipo **1-a-Molti**:

- Da **Fornitore** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Prodotto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Progetto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)

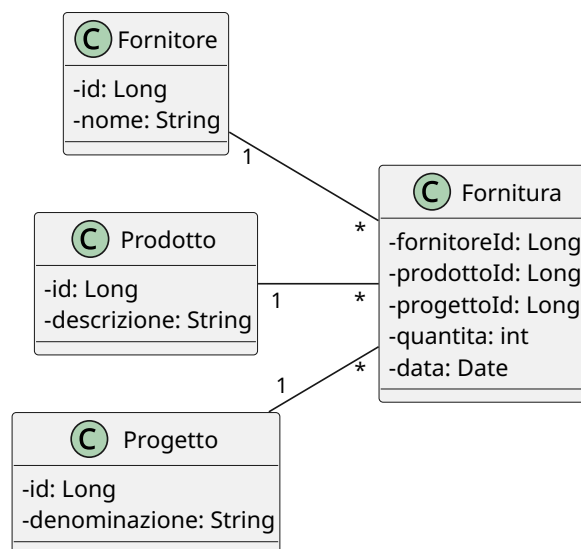


Figura 1.5: Reificazione dell'associazione ternaria.

Nella realtà, gli eventi n-ari catturano quasi sempre proprietà quantitative. Riprendendo l'esempio precedente, l'atto della fornitura genera intrinsecamente una *quantità* di pezzi consegnati e una *data* di consegna. Questi attributi non appartengono a nessuna delle tre classi isolate, ma alla loro combinazione. In UML si modella una **Classe di Associazione** (es. **Fornitura**) collegata tramite una linea tratteggiata al rombo dell'associazione ternaria.

Il processo di esplicitazione in questo scenario è immediato: la classe di associazione **Fornitura** viene promossa a classe centrale del cluster, inglobando gli attributi **quantità** e **data**. Le tre classi originarie si collegano a essa tramite relazioni binarie dirette.

⚠ La trappola delle cardinalità ternarie

Nelle associazioni ternarie UML, la cardinalità posta vicino a una classe (es. **Progetto**) indica quante istanze di quella classe possono combinarsi con una **coppia fissa** di istanze delle altre due classi (**Fornitore** e **Prodotto**). Questa logica è controintuitiva e differisce dai diagrammi ER classici (notazione di Chen), inducendo spesso i progettisti in errore. Sostituire subito la ternaria con una classe



esplicitata azzera ogni ambiguità interpretativa.

1.1.6 Aggregazione e Composizione

Nello sviluppo dei Class Diagrams, le associazioni standard indicano un legame logico simmetrico. Tuttavia, per modellare scenari in cui esiste un rapporto di possesso o di subordinazione strutturale, l'UML introduce le **relazioni tutto-parte** (part-whole relationships). Queste si dividono in due varianti fondamentali, distinte in base al vincolo sul ciclo di vita degli oggetti coinvolti: l'aggregazione e la composizione.

L'**aggregazione** rappresenta una relazione in cui un oggetto “Tutto” (aggregante) contiene o colleziona uno o più oggetti “Parte” (aggregati), ma questi ultimi mantengono un'esistenza autonoma all'interno del dominio. Si tratta di un legame “*has-a*” ad accoppiamento debole.

- **Sintassi UML:** Si rappresenta graficamente con una linea che termina con un **rombo vuoto** (bianco) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** La distruzione dell'oggetto aggregante **non** comporta la distruzione degli oggetti aggregati.

Esempio 1.2 (Università e Professore). Si consideri la classe **Università** (il Tutto) e la classe **Professore** (la Parte). Un'università raccoglie un insieme di docenti. Tuttavia, se l'università dovesse cessare la sua attività, i singoli professori non verrebbero eliminati dal sistema: essi continuerebbero ad esistere come entità autonome e potrebbero essere associati ad altri atenei.

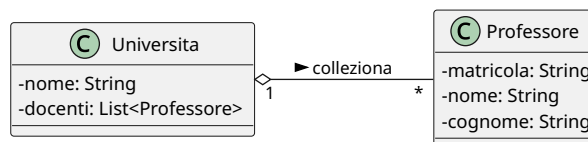


Figura 1.6: Esempio di aggregazione

La **composizione** è una forma di aggregazione più stringente, esclusiva e gerarchica. In questo scenario, l'oggetto “Parte” è una componente indissolubile del “Tutto”. Una parte può appartenere a un solo tutto alla volta e non ha alcuna autonomia esistenziale.

- **Sintassi UML:** Si rappresenta con una linea che termina con un **rombo pieno** (nero) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** Il ciclo di vita delle parti è totalmente subordinato a quello del tutto. Di conseguenza, la distruzione dell'oggetto composto (Tutto) determina l'istanza di **distruzione automatica** di tutte le sue parti. La cardinalità dal lato del tutto è rigorosamente 1 o 0..1.

Esempio 1.3 (Ordine ed Elemento d'Ordine). Si consideri un sistema di e-commerce con le classi **Ordine** (il Tutto) e **RigaOrdine** (la Parte, che descrive il prodotto e la quantità). Una riga d'ordine ha significato logico solo se contestualizzata all'interno dello specifico acquisto. Se l'utente decide di eliminare l'intero **Ordine**, il sistema deve rimuovere a cascata tutte le relative istanze di **RigaOrdine**.

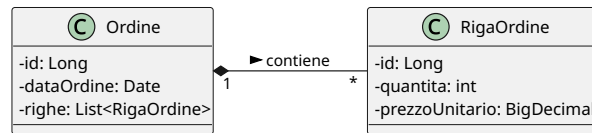


Figura 1.7: Esempio di composizione

🔍 L'aggregazione non ha alcuna necessità tecnica stringente: una semplice associazione fa esattamente lo stesso identico lavoro. A livello di tabelle, sia un'associazione generica sia un'aggregazione si tradurranno nello stesso modo, la differenza è puramente semantica e documentale, non implementativa. Alcuni autorevoli ingegneri del software (tra cui Martin Fowler, uno dei padri dell'architettura software moderna) sostengono da anni che l'aggregazione UML sia fondamentalmente inutile perché la linea di demarcazione con l'associazione standard è troppo sfumata e crea più ambiguità che benefici. Fowler la definisce ironicamente "un effetto placebo grafico".

Il caso della composizione è invece più significativo, perché il diagramma fornisce un'informazione in più: la parte non può esistere senza il tutto.

1.1.7 Generalizzazioni/Specializzazioni

L'ereditarietà modella una relazione di tipo “IS-A” (è-un) tra una classe più generale, detta **Generalizzazione** (o Superclasse, per analogia con i contesti OOP), e una o più classi più specifiche, dette **Specializzazioni** (o Sottoclassi).

Le specializzazioni ereditano automaticamente tutti gli attributi, i ruoli e le associazioni della generalizzazione, senza necessità di ridefinirli, e possono estendere il modello introducendo attributi o associazioni propri.

Sintassi UML: Si rappresenta graficamente con una linea che termina con una **freccia con la punta triangolare vuota** rivolta verso la generalizzazione.

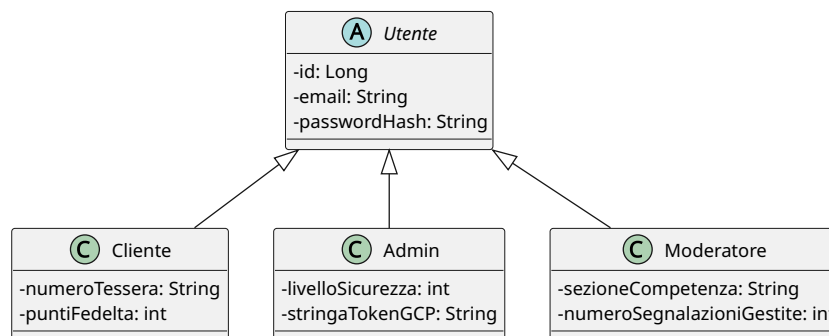


Figura 1.8: Esempio di ereditarietà

È fondamentale comprendere che **l'ereditarietà non esiste nel modello relazionale delle basi di dati** (o almeno, non secondo il modello relazionale dei dati che noi studieremo). I database relazionali non possiedono strutture gerarchiche native; essi conoscono esclusivamente il concetto di tabella (relazione), che è per sua natura una struttura “piatta” e bidimensionale.

La generalizzazione appartiene quindi in modo esclusivo al livello della **progettazione concettuale**. In questa fase, lo scopo del Class Diagram è modellare il dominio reale nel modo più fedele e naturale possibile, catturandone la semantica e i vincoli logici

(ovvero i rapporti di sotto-tipo/sopra-tipo) ed eliminando qualsiasi vincolo tecnologico. Il problema di come “appiattare” questa gerarchia per adattarla a un database relazionale sarà l’obiettivo centrale della successiva fase di progettazione logica.

Una generalizzazione può essere classificata combinando due tipi di vincoli ortogonali, utili a descrivere la natura delle istanze nel dominio reale:

1. **Completezza (Totale vs Parziale):**

- **Totale (Complete):** Ogni istanza della superclasse deve obbligatoriamente appartenere ad almeno una delle subclassi. Non esistono elementi “generici” (corrisponde al concetto di *Classe Astratta* in OOP).
- **Parziale (Incomplete):** Un’istanza della superclasse può esistere senza dover per forza appartenere a una delle subclassi catalogate.

2. **Disgiunzione (Disgiunta vs Sovrapposta):**

- **Disgiunta (Disjoint):** Un’istanza della superclasse può appartenere al massimo a **una sola** delle subclassi. Le sottoclassi sono mutuamente esclusive.
- **Sovrapposta (Overlapping):** Una singola istanza della superclasse può appartenere contemporaneamente a più subclassi distinte.

Ci sono dunque quattro combinazioni possibili. Nel diagramma UML, i vincoli della generalizzazione si indicano inserendo un’etichetta testuale racchiusa tra parentesi graffe (*Generalization Set*) in prossimità delle linee di ereditarietà, specificando la coppia di proprietà (es: {complete, overlapping}).

Esempio 1.4 (Esempi di Vincoli). Si considerino i seguenti scenari aziendali:

- **Totale / Disgiunta:** La superclasse *ContoBancario* ha come subclassi *ContoCorrente* e *ContoDeposito*. Un conto deve essere obbligatoriamente di un tipo o dell’altro, e non può essere entrambe le cose contemporaneamente.
- **Parziale / Sovrapposta:** La superclasse *Persona* ha come subclassi *Studente* e *Dipendente*. All’interno dell’anagrafica universitaria, una persona può essere solo un cittadino esterno (parziale), oppure può essere contemporaneamente uno studente che lavora per l’ateneo (sovrapposta).

1.2 La Documentazione di Supporto: I Dizionari dei Dati

Il Class Diagram, pur essendo uno strumento visivo formidabile, non è espressivamente sufficiente a catturare l’intera semantica di un dominio applicativo. Elementi come il significato preciso di un attributo, il contesto d’uso di un’associazione o le regole aziendali complesse (*Business Rules*) non possono essere espressi tramite nodi e archi.

La progettazione concettuale si completa quindi attraverso i **Dizionari dei Dati**, estensioni testuali strutturate che definiscono formalmente ogni componente dello schema concettuale. Questi documenti rappresentano la specifica dei requisiti su cui si baserà lo sviluppo dello strato di persistenza e della logica di business.

I dizionari dei dati sono:

- **Dizionario delle Classi:** descrive in modo univoco ogni entità del sistema. Si presenta come una tabella in cui ogni riga rappresenta una classe. Per ogni classe vengono forniti il nome (prima colonna), una descrizione in linguaggio naturale per evitare ambiguità terminologiche (seconda colonna), nome, tipo e descrizione di ciascun attributo (terza colonna).
- **Dizionario delle Associazioni:** si presenta come una tabella analoga alla precedente, in cui ogni riga rappresenta un’associazione. Per ogni associazione vengono

forniti il nome (prima colonna), descrizione (seconda colonna), le classi coinvolte con ruolo, molteplicità e descrizione (terza colonna).

- **Dizionario dei Vincoli:** è il documento più critico. I **Vincoli di Integrità Esogeni** (o regole di business) sono restrizioni sul dominio dei dati che **non possono** essere espresse graficamente nel Class Diagram tramite la normale sintassi UML. Mentre una cardinalità $1..*$ esprime un vincolo strutturale grafico, una regola del tipo “*Un cliente non può effettuare un ordine se il suo account è sospeso*” non ha un simbolo UML dedicato. Viene quindi codificata nel dizionario dei vincoli in linguaggio naturale o tramite espressioni logico-matematiche formali (come l’OCL - *Object Constraint Language*). Si presenta come una tabella a due colonne in cui ogni riga rappresenta un vincolo, nella prima colonna si inserisce il nome del vincolo oppure un suo identificatore univoco (es. BR - Business Rule), nella seconda colonna la sua descrizione in linguaggio naturale.

Esempio 1.5 (Catalogo dei Vincoli d’Integrità).

BR01 (Coerenza Temporale): La data di spedizione di un `Ordine` deve essere strettamente successiva o uguale alla sua data di creazione.

BR02 (Integrità Finanziaria): L’attributo `totale` di un `Ordine` deve essere matematicamente pari alla sommatoria del prodotto tra `quantita` e `prezzoUnitario` di tutte le istanze di `RigaOrdine` a esso collegate.

BR03 (Regola di Business): Un `Utente` con ruolo `Admin` non può associare a sé stesso un’istanza di `Ordine` in veste di `acquirente`.



Nel dizionario dei vincoli non vanno inseriti vincoli che sono già espressi nel class diagram.

I vincoli devono garantire la qualità interna dei dati nella base di dati.

Modello Relazionale e Progettazione Logica

2.1 Introduzione

Il passaggio dalla progettazione concettuale alla progettazione logica rappresenta la transizione dal *cosa* deve essere rappresentato al *come* organizzare concretamente i dati all'interno di un sistema di gestione di basi di dati (DBMS). Questo processo richiede l'adozione di uno specifico **modello dei dati**.

Sebbene la storia dell'informatica abbia visto l'evoluzione di diversi paradigmi — come il modello gerarchico, quello reticolare e quello orientato agli oggetti —, la nostra trattazione si concentrerà sul **modello relazionale**, che costituisce lo standard industriale prevalente. Vale la pena notare che i DBMS moderni più diffusi (come PostgreSQL e Oracle) adottano un'evoluzione di questo approccio, denominata **modello relazionale a oggetti**, la quale estende le fondamenta del modello relazionale puro integrando costrutti tipici del paradigma OOP.

Uno dei principali vantaggi nell'aver adottato i **Class Diagrams UML** nella fase concettuale risiede nella loro assoluta indipendenza tecnologica: essi si adattano teoricamente a qualunque modello logico finale. Di contro, questa spiccata espressività introduce un inevitabile **disallineamento semantico** (*impedance mismatch*) quando l'obiettivo è la traduzione in uno schema relazionale puro.

Nel passaggio alla progettazione logica relazionale, le principali sfide strutturali da affrontare saranno dettate dai rigidi vincoli di atomicità e piattezza del dato tipici delle tabelle:

- **Attributi strutturati:** Il modello relazionale esige che i valori degli attributi siano atomici e non strutturati. Non è quindi possibile nidificare record, oggetti o tipi di dato complessi all'interno di una colonna.
- **Attributi a valore multiplo (multivalore):** Ogni cella di una tabella relazionale può contenere esclusivamente un singolo valore atomico, vietando l'uso nativo di collezioni, array, liste o set all'interno di un singolo attributo.
- **Assenza di ereditarietà:** Il modello relazionale puro non supporta nativamente le gerarchie di generalizzazione-specializzazione, costringendo il progettista ad applicare strategie di “appiattimento” o di scomposizione strutturale.

2.2 Concetti Fondamentali

Il **modello relazionale** si basa sul concetto matematico di relazione. Questo modello rappresenta la struttura logica e piatta su cui verranno mappati i dati dell'applicazione. Per formalizzare rigorosamente il modello, è necessario definire i suoi tre componenti atomici: i domini, lo schema e la relazione stessa.

Un **dominio** (indicato con D) è un insieme di valori atomici, ovvero non ulteriormente scomponibili. Ogni dominio è caratterizzato da un nome e da un tipo di dato (es: `int`, `varchar`, `date`).

Osservazione 2.1 (Il Tipo di Dato come Dominio Implicito). Nella transizione dalla teoria all'ingegneria del software, il **tipo di dato** (es. `int`, `varchar`, `date`) definisce l'insieme universo di partenza, agendo come **dominio massimo implicito** dell'attributo.

Eventuali restrizioni logiche (come un *range* di valori validi per un voto universitario o un'età) si configurano come un **sottoinsieme** di tale dominio base. Vedremo in seguito come vengono implementate limitazioni specifiche.

Uno **schema relazionale** (o schema di tabella) definisce la struttura logica dei dati. Esso è costituito da un nome R e da un insieme di attributi $A_1, A_2, \dots, A_n$, a ciascuno dei quali è associato un dominio D_i , che denoteremo anche con $dom(A_i)$.

Lo schema viene denotato formalmente come:

$$R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$$

Lo schema relazionale corrisponde dunque esattamente al concetto di **Classe**. Esso definisce la struttura dei dati, i nomi dei campi e i rispettivi tipi, agendo come un “blueprint” statico.

Matematicamente, date le premesse precedenti, definiamo il dominio totale dello schema come il prodotto cartesiano dei domini dei singoli attributi:

$$D_{tot} = D_1 \times D_2 \times \dots \times D_n$$

Una **relazione** r sullo schema R è un sottoinsieme finito del prodotto cartesiano dei domini degli attributi:

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

Gli elementi che compongono questo sottoinsieme sono detti **tuple** (o *n*-uple). Una *tuple* è un vettore di valori $(v_1, v_2, \dots, v_n)$ tale che ogni $v_i \in D_i$.



La corrispondenza con OOP è la seguente:

- Una **Tupla** corrisponde al singolo **Oggetto** (una istanza della classe contenente dati reali).
- La **Relazione** corrisponde all'insieme di tutti gli oggetti di quella classe **attualmente persistiti** nel database in un determinato istante temporale (lo stato corrente della tabella).

Esempio 2.1 (Formalizzazione di un'Anagrafica). Definiamo lo schema relazionale per la classe **Studente**:

$$STUDENTE(\text{matricola} : \text{String}, \text{nome} : \text{String}, \text{eta} : \text{Integer})$$

Il prodotto cartesiano di questi tre domini genererebbe infinite combinazioni (comprese matricole inesistenti accoppiate a nomi casuali ed età assurde).

La **relazione** $r(STUDENTE)$ conterrà invece solo le tuple degli studenti reali inseriti nel sistema:

$$r = \{('654321', 'Giovanni', 22), ('123456', 'Mario', 24)\}$$

Osservazione 2.2. È significativo evidenziare che una relazione è *un insieme*, dunque non ammette duplicati e non rappresenta una struttura ordinata.

In sintesi:

Nel passaggio dalla progettazione concettuale alla modellazione logica, i concetti del paradigma orientato agli oggetti (*OOP*) e del Class Diagram si traducono come segue:

- **Classi** → **Schemi Relazionali**: Le classi si traducono in schemi di relazione, identificati da un nome (corrispondente al nome della classe) e da un insieme di attributi.
- **Attributi** → **Domini**: A ciascun attributo dello schema è associato un **dominio**, ovvero l'insieme di tutti i possibili valori atomici che quell'attributo può assumere (equivalente al tipo di dato in programmazione).
- **Istanze** → **Relazione (Stato)**: Una **relazione** è un sottoinsieme discreto e finito del dominio totale (il prodotto cartesiano di tutti i singoli domini). Essa rappresenta l'insieme delle tuple (istanze o oggetti) effettivamente memorizzate e persistite nel sistema in un dato momento temporale.

2.3 Le Chiavi

Fino ad ora abbiamo analizzato come tradurre classi e attributi del Class Diagram nel modello relazionale, lasciando in sospeso la traduzione delle associazioni. Per poter modellare le relazioni tra tabelle, è prima necessario introdurre il concetto di chiave.

Sia $R(A_1, A_2, \dots, A_n)$ uno schema di relazione. Un sottoinsieme di attributi $SK \subseteq \{A_1, A_2, \dots, A_n\}$ si definisce **superchiave** se i suoi valori identificano in modo univoco ogni singola tupla all'interno di qualsiasi istanza valida della relazione.

Esempio 2.2 (Superchiavi di un'Anagrafica). Si consideri lo schema:

STUDENTE(CF, matricola, nome, cognome, dataNascita)

I singoletti $\{CF\}$ e $\{matricola\}$ sono superchiavi. Di conseguenza, **qualsiasi sottoinsieme che li contenga** (es: $\{CF, nome\}$ o $\{matricola, cognome\}$) è a sua volta una superchiave.

Osservazione 2.3 (Ipotesi di Esistenza). Nella fase di progettazione concettuale si assume l'ipotesi che ogni istanza di una classe sia distinguibile dalle altre. Questa proprietà ci garantisce che **esiste sempre almeno una superchiave** per ogni schema relazionale: nella peggiore delle ipotesi (assenza di identificatori naturali), la superchiave coinciderà con l'insieme di tutti gli attributi dello schema.

Un sottoinsieme di attributi K è una **chiave candidata** se è una **superchiave minimale**. Dire che una superchiave è *minimale* significa che è **irriducibile**: eliminando un qualsiasi attributo da K , l'insieme rimanente perde la proprietà di univocità e cessa di essere una superchiave.

Questo **non** significa che debba avere il numero minimo di attributi in assoluto (cardinalità minima). Se nello schema coesistono una superchiave irriducibile di cardinalità 1 e una di cardinalità 2, sono entrambe, con pari dignità, chiavi candidate.

È evidente che una superchiave composta da un unico attributo (un singoletto) è di fatto e necessariamente una chiave candidata.

Esempio 2.3 (Chiave Candidata Composta). Si consideri lo schema che mappa la classe degli esami universitari:

ESAME(matricola, corso, voto, data)

In questo scenario, il sottoinsieme $\{\text{matricola}, \text{corso}\}$ rappresenta una chiave candidata (di fatto, è l'unica chiave candidata in questo caso). Essa è minimale perché se eliminassimo **corso**, la sola **matricola** non garantirebbe l'univocità (uno studente può sostenere più esami); viceversa, eliminando **matricola**, il solo **corso** conterrebbe i voti di tutti gli studenti.

Poiché in uno schema relazionale possono coesistere più chiavi candidate, il progettista deve sceglierne una da eleggere come identificatore ufficiale delle tuple. La chiave candidata scelta prende il nome di **Chiave Primaria** (*Primary Key* - PK).

La selezione della chiave primaria è una decisione puramente ingegneristica e architettonale. Nello schema relazionale scritto, la chiave primaria si indica **sottolineando** gli attributi che la compongono.

Esempio 2.4 (Scelta della PK). Riprendendo lo schema **STUDENTE**, le chiavi candidate sono CF e **matricola**. Sceglieremo come chiave primaria la **matricola**:

STUDENTE(CF, matricola, nome, cognome)

La **Chiave Esterna** (*Foreign Key* - FK) è lo strumento fondamentale del modello relazionale per tradurre le associazioni.

Siano R_1 e R_2 due schemi relazionali (non necessariamente distinti). Un insieme di attributi FK di R_1 è una chiave esterna che punta a R_2 se:

1. Gli attributi in FK sono in corrispondenza biunivoca con gli attributi che compongono la **chiave primaria** (PK) di R_2 .
2. I domini degli attributi corrispondenti sono identici ($dom(FK_i) = dom(PK_i)$).

Il vincolo impone che per ogni tupla t_1 nell'istanza corrente di R_1 , il valore di $t_1[FK]$ deve essere nullo (NULL), oppure coincidere esattamente con il valore $t_2[PK]$ di una tupla t_2 attualmente esistente nell'istanza di R_2 .

Nello schema testuale, la presenza di una chiave esterna si denota convenzionalmente con una **doppia sottolineatura**.

2.4 Traduzione delle Associazioni

2.4.1 Associazioni 1 a 1

Un'associazione **1 a 1** tra due classi A e B indica che a un'istanza di A corrisponde al massimo un'istanza di B , e viceversa. Nel modello relazionale, questo legame si traduce mantenendo due schemi di relazione separati (due tabelle) e introducendo una **Chiave Esterna (FK)** in uno dei due schemi che punta alla Chiave Primaria (PK) dell'altro.

Affinché la relazione rimanga rigorosamente 1 a 1, non è sufficiente definire la chiave esterna. Se ci limitassimo a questo, la tabella che ospita la FK potrebbe avere più tuple che puntano alla stessa PK della tabella bersaglio, trasformando di fatto la relazione in una 1 a N.

Per garantire la biunivocità dell'associazione, è obbligatorio imporre un **vincolo di univocità** sugli attributi che compongono la chiave esterna. Matematicamente, se lo schema R_1 contiene la chiave esterna FK verso R_2 , deve valere che per ogni coppia di tuple distinte $t_x, t_y \in r(R_1)$:

$$t_x[FK] \neq t_y[FK] \quad (\text{escludendo i valori NULL})$$

La scelta di quale delle due tabelle debba ospitare la FK dipende dalle **cardinalità minime** (l'opzionalità o obbligatorietà dell'associazione):

1. **Partecipazione Obbligatoria vs Opzionale ($0..1 \leftrightarrow 1..1$):** Se la classe A partecipa in modo opzionale ($0..1$) e la classe B in modo obbligatorio ($1..1$), la chiave esterna **deve essere inserita nello schema** B . In questo modo, la FK in B potrà essere dichiarata **NOT NULL** (vincolo di esistenza garantito). Se la mettessimo in A , saremmo costretti ad ammettere valori **NULL** per le istanze di A che non partecipano all'associazione, sprecando spazio e riducendo l'eleganza relazionale.
2. **Entrambe Obbligatorie ($1..1 \leftrightarrow 1..1$):** Se entrambe le partecipazioni sono rigide, la FK può essere posizionata indifferentemente in A o in B . (In alcuni casi estremi di forte coesione, le due tabelle possono persino essere fuse in un unico schema relazionale).

Esempio 2.5 (Direzione di un Dipartimento). Si consideri l'associazione 1 a 1 tra **Professore** ($0..1$) e **Dipartimento** ($1..1$): un professore può dirigere al massimo un dipartimento, ma un dipartimento deve avere esattamente un direttore.

Seguendo la regola, posizioniamo la chiave esterna nel **Dipartimento**:

- **PROFESSORE**(codiceDocente, nome, cognome)
- **DIPARTIMENTO**(codice, nome, codiceDirettore)

Vincoli su **DIPARTIMENTO.direttore**:

- **Foreign Key:** punta a **PROFESSORE.matricola**
- **UNIQUE:** per garantire che lo stesso professore non diriga due dipartimenti (vincolo 1:1).
- **NOT NULL:** per garantire che ogni dipartimento abbia un direttore (cardinalità minima 1).

Bozze

A.1 Aspetti implementativi dell'ereditarietà

Poiché il modello relazionale delle basi di dati si basa su tabelle bidimensionali e non prevede il concetto nativo di ereditarietà, lo sviluppatore backend deve scegliere come tradurre la gerarchia UML in tabelle SQL. Esistono tre strategie standard, ognuna con precisi trade-off ingegneristici.

Immaginiamo la gerarchia: **Utente** (id, email) → **Cliente** (voti) e **Admin** (livelloAccesso).

1. Tabella Unica (Single Table Inheritance)

La gerarchia viene collassata in **un'unica grande tabella** che contiene tutti gli attributi della superclasse e delle subclassi. Viene aggiunta una colonna speciale detta **Discriminator** (es. `tipo_utente`) per capire quale riga rappresenti quale classe.

- **Vantaggi:** Performance massime. Le query non richiedono alcuna operazione di JOIN per recuperare i dati specifici.
- **Svantaggi:** Violazione della normalizzazione. Gli attributi specifici delle subclassi (es. `voti` per l'admin o `livelloAccesso` per il cliente) devono obbligatoriamente accettare valori NULL. Se le subclassi hanno molti attributi unici, la tabella diventa estremamente sparsa e spreca spazio.

2. Tabella per Classe Concreta (Table Per Class Inheritance)

Viene creata una tabella indipendente per ogni sottoclasse concreta. Gli attributi della superclasse vengono **duplicati** in ciascuna tabella figlia. Non esiste una tabella per la superclasse.

- **Vantaggi:** Struttura pulita se si interrogano sempre e solo le classi specifiche (es. cerco solo i clienti). Nessun valore NULL forzato.
- **Svantaggi:** Le query polimorfiche (es. "Seleziona tutti gli utenti del sistema, indipendentemente dal tipo") sono un incubo computazionale: richiedono pesanti operazioni di UNION tra tutte le tabelle. Inoltre, non è possibile definire una Foreign Key che punti genericamente a un **Utente**.

3. Tabelle Congiunte (Joined / Table Per Subclass Inheritance)

Viene creata una tabella per la superclasse e una tabella distinta per ciascuna subclassi. Le tabelle delle subclassi contengono **solo** i propri attributi specifici e una chiave primaria che funge contemporaneamente da **Chiave Esterna (FK)** verso la tabella padre.

- **Vantaggi:** Massimo rigore relazionale e pulizia matematica (rispetta la normalizzazione). Nessun valore NULL inutile, nessuna duplicazione di colonne.
- **Svantaggi:** Calo drastico delle performance su gerarchie profonde. Ogni singola lettura di una sottoclasse richiede un'operazione di JOIN tra la tabella padre e la tabella figlia.

A.2 lezione 7

Dalla progettazione concettuale alla progettazione logica. In mezzo c'è la scelta del modello dei dati. Noi seguiremo il modello dei dati relazionale dei dati. Ci sono anche altri modelli: gerarchico, ad oggetti, relazionale ad oggetti. Ad esempio Oracle e Postgres (che sono i sistemi più utilizzati) usano quest'ultimo modello.

Uno dei vantaggi della scelta del class diagram per la progettazione concettuale è che si adatta bene a qualsiasi tipo di modello dei dati. Lo svantaggio è che pone alcuni problemi da risolvere nel passaggio dalla progettazione concettuale a quella logica. In questo passaggio, nel caso di modello dei dati relazionale, le principali difficoltà saranno:

- attributi strutturati: il modello relazionale dei dati esige valori non strutturati per gli attributi.
- attributi a valore multiplo: il modello relazionale dei dati esige un valore unico.
- il modello relazionale non supporta le gerarchie generalizzazione-specializzazione.